

UnionLabs SDR Platform

Contents

UnionLabs SDR Platform	1
The three questions every run answers	1
Quick start	2
Layout (detail in docs/STRUCTURE.md)	2
The physical layers	2
Documentation	3
Install	3

UnionLabs SDR Platform

Write an algorithm once; run it over **any** physical layer. Your algorithm says only *what to transmit* and *what to receive* — the platform carries it over a USRP software-defined radio, a LoRa module, or no radio at all, without a single line of your code changing.

```
./run.sh --algo fl # federated learning, no radio needed
./run.sh --algo fl --channel lora --lora-sf 9 # the SAME algorithm, over LoRa
./run.sh --algo fl --channel usrp --snr-db 8 # the SAME algorithm, over the USRP modem
./run.sh list # what algorithms exist, and their roles
```

New here? Start with [docs/BEGINNER_GUIDE.md](#).

The three questions every run answers

Every experiment is a choice of three independent things. Mixing them up is the most common source of confusion, so the flags keep them apart:

Question	Flag	Choices
What am I running?	--algo	any folder in experiments/ (./run.sh list)
Which PHY carries it?	--channel	ideal · usrp · lora
Which part am I?	--role	loopback chain gossip multi · tx rx relay peer

The first four roles build the **whole network in one process** — that is how you develop. The last four make this process **one node** of it — that is how you deploy.

A fourth flag says **how** the chosen PHY is attached — --usrp-backend pyphy|radio, --lora-backend sim|serial|spi. Every PHY's default backend needs **no hardware**, so you can develop the entire experiment on a laptop and then change one flag to move to radios.

```
# one process, whole network, no hardware — how you develop
./run.sh --algo dl --role gossip --agents 6 --topology ring --channel lora

# one terminal (or one computer) per node — how you deploy
```

```
./run.sh --algo dl --node 0 --agents 3 --radio serial=30CD424
./run.sh --algo dl --node 1 --agents 3 --radio serial=30CD3F7
```

Quick start

```
./run.sh                # defaults: algo=echo, role=loopback, channel=ideal
./run.sh --algo marl    # any algorithm from experiments/
./run.sh --help         # every option, grouped
./radio.sh rx          # raw receive on a USRP (B210 default; --device n210|x310)
```

Layout (detail in [docs/STRUCTURE.md](#))

Four files and five folders at the top level — everything you write lives in **one** of them:

```
README.md              what this is (you are here)
HOW_TO_ADD_ALGORITHM.md the tutorial: write your own experiment
run.sh                 run an experiment over a PHY
radio.sh                raw TX/RX on a USRP
```

```
experiments/  <- THE folder you work in. One subfolder per experiment, each with app.py
union/        the UnionLabs bridge: one contract for every PHY and testbed
drivers/      the PHYs: usrp/ (C++ modem + pyphy), lora/ (SX1276), sim/
docs/         every guide, reference and PDF
deploy/       Docker + install
results/      generated output (gitignored)
```

Path	What
experiments/	your work, one folder each
union/	the middleware — phy_link.py, run_algo.py, driver.py
drivers/	the driver layer , one per PHY — usrp/, lora/, sim/
docs/	guides, references, PDFs, diagrams, slides

Shipped experiments: echo plain_echo fl dl marl marl_multi clip_semcom stc_aircomp jammer — plus _template/ to copy, and _shared/ for libraries that more than one experiment uses.

Two layers. union/ is the abstraction, shared by everything. drivers/<name>/ is one driver per (PHY × testbed). Experiments import from union/ only, which is why the same experiment runs on any driver — the portability POWDER and AERPAW do not expose.

The physical layers

--channel	Driver	Backends
ideal	drivers/sim	—
usrp	drivers/usrp	pyphy (default, no radio) · radio
lora	drivers/lora	sim (default) · serial · spi

- **ideal** — lossless and in-process. Check your logic here first.
- **usrp** — the real C++ modem: OFDM or single-carrier, LDPC/turbo/Viterbi FEC, sync, CFO.
- **lora** — SX1276. The 255-byte MTU means the driver fragments and retransmits, and it reports the real airtime that cost.

Each PHY keeps its own knobs, because the PHYs genuinely differ — we assemble the USRP waveform, while a LoRa chip embeds its modulation and CRC:

```

shared  --freq (MHz)  --max-attempts  --arq
usrp    --modulation --fec --samp-rate --symbol-rate --tx-gain --rx-gain
        --ack-transport tcp|rf  --ack-timeout  --radio serial=...|addr=...
lora    --lora-sf 7..12  --lora-cr 5..8  --lora-bw 125000|250000|500000
        --lora-power  --lora-port /dev/ttyUSB0

```

Passing one PHY's knob while another is selected prints a NOTE rather than being silently ignored.

Documentation

Doc	Its one job
docs/BEGINNER_GUIDE.md	start here — install, first run, every run.sh option explained
HOW_TO_ADD_ALGORITHM.md	write your own app.py: the contract, roles, linking existing code
docs/STRUCTURE.md	the repo map and how the two layers stack
drivers/usrp/GUIDE.md	the USRP PHY: every way to drive it directly
drivers/lora/README.md	the LoRa PHY: firmware, wiring, the three attachments
docs/PARAMETERS.md	every C++ modem option (auto-generated, always current)
docs/COMMANDS.md	ready-to-run commands: algorithm-level, then raw TX/RX per modulation scheme
docs/USRP_CARRIER_MODULATION.txt	raw commands per device (B210 / N210 / X310)
docs/SYSTEM_REFERENCE.pdf	the deep engine reference — the math and every algorithm
docs/HARDWARE.md	hardware setup and radio inventory
docs/MANIFEST.md	file index and the CLI <-> sdr.py mapping

Install

```

deploy/initialization.sh --build      # toolchain + compile drivers/usrp/build/sdr_system

```

--minimal skips torch/networkx/opencv (PHY only); --docs adds the PDF toolchain. See deploy/DOCKER.md to run it all in a container instead.